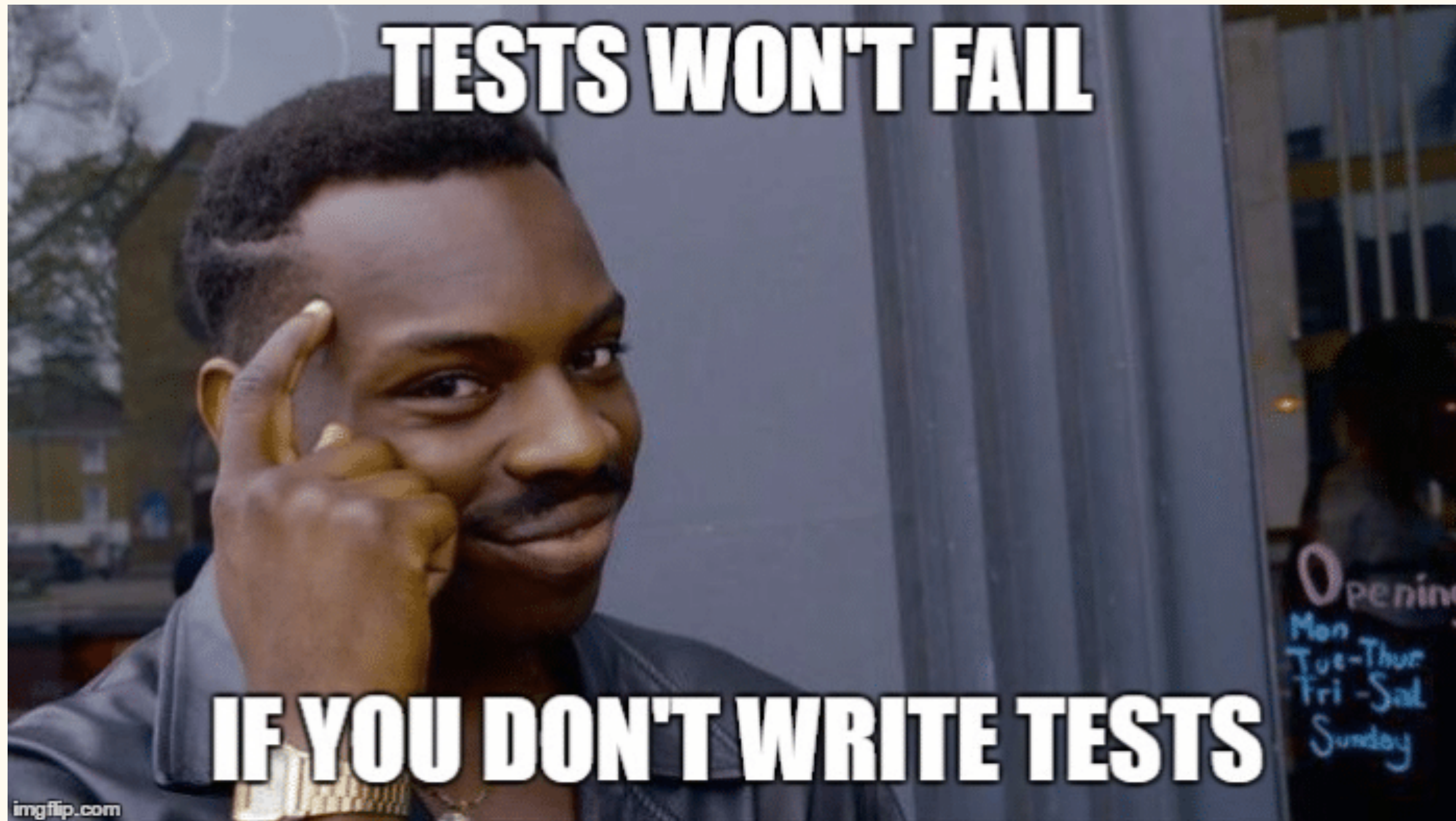


TESTING YOUR CODE





I. Lesson

A simple test module



```
#[cfg(test)]
mod tests {
    #[test]
    fn it_works() {
        assert_eq!("hello world!", "hello world!");
    }
}
```


How cargo test works

How `cargo test` Works

1. Cargo reads project metadata

- Reads `Cargo.toml` and dependencies.

2. Compile library code

- Compiles `src/lib.rs` and dependencies.

3. Compile test code

- **Unit Tests:** Inside `#[cfg(test)]` modules in library/binary files.
- **Integration Tests:** Each file in `tests/` is compiled as a separate executable.

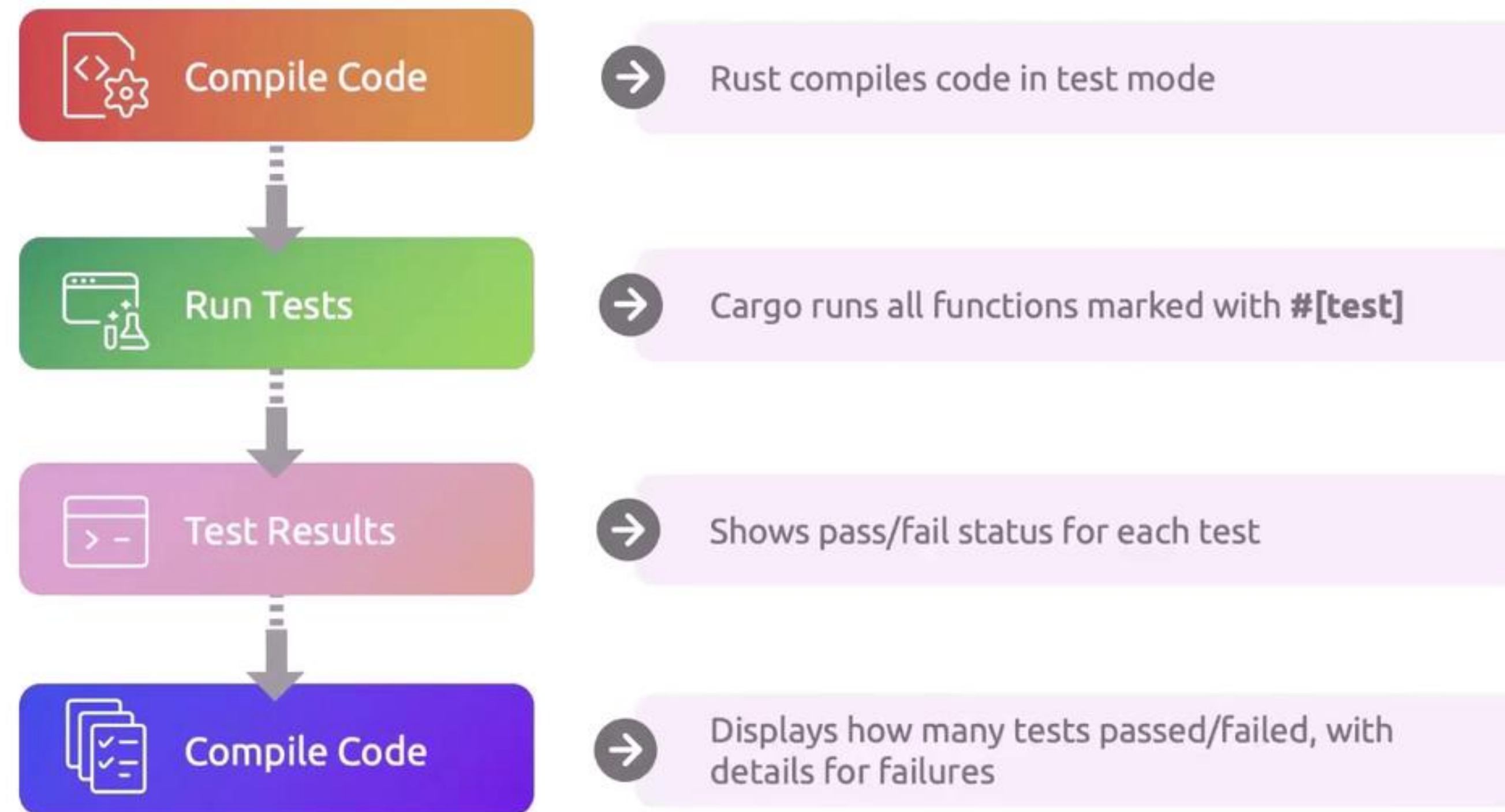
4. Link test binaries

- Test code is linked with the compiled library.

5. Run tests

- Executes all test binaries.
- Reports passed/failed results.

Understanding Test Output



docs.rs



Type 'S' or '/' to search

Search

I'm Feeling Lucky

Recent Releases

spacetimedb-primitives-1.8.0	Primitives such as TableId and ColumnIndexAttribute	14 seconds ago
spacetimedb-memory-usage-1...	Provides the trait `MemoryUsage`	38 seconds ago
keyboard-layout-lib-0.1.0	Cross-platform keyboard layout utilities	one minute ago
octokit-rs-0.1.15	Octokit RS is a lib with the up-to-date Github types	one minute ago
espresso-logic-3.1.2	Rust bindings for the Espresso heuristic logic minimizer (UC Berkeley)	5 minutes ago
qtrace-0.1.0	QEMU user space tracing	7 minutes ago
anychain-ethereum-0.1.34	A Rust library for Ethereum-focused cryptocurrency wallets, enabling seamless transactions on the Eth...	7 minutes ago
neighborgood-0.0.0	Safer social media app idea.	8 minutes ago
makiatto-cli-0.4.3	CLI tool for managing Makiatto CDN deployments	8 minutes ago
copy_impl-0.3.3	Macro for effortlessly duplicating impl block code across various types in Rust.	9 minutes ago
storm-config-0.27.3	A crate containing the configuration structure and utilities used by Storm Software monorepos.	10 minutes ago
storm-workspace-0.19.27	A crate containing various utilities and tools used by Storm Software to manage monorepos.	10 minutes ago

II. Project

Example with divide function

```
fn divide(a: i32, b: i32) -> Result<i32, String> {  
    if b == 0 {  
        return Err("Cannot divide by zero".to_string());  
    }  
    Ok(a / b)  
}
```

↳ We need to make sure this works!

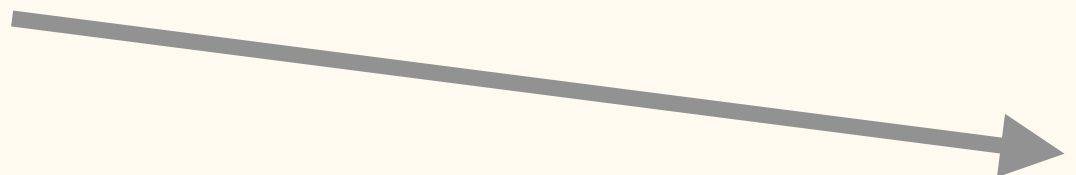


```
#[cfg(test)]  
mod tests {  
    use super::*;  
  
    #[test]  
    fn test_valid_division() {  
        let result = divide(10, 2);  
        assert_eq!(result, Ok(5));  
    }  
  
    #[test]  
    fn test_division_by_zero() {  
        let result = divide(10, 0);  
        assert!(result.is_err());  
    }  
  
    #[test]  
    fn test_negative_numbers() {  
        let result = divide(-10, 2);  
        assert_eq!(result, Ok(-5));  
    }  
}
```

Test panicking

```
fn get_item(index: usize) -> i32 {  
    let items = vec![10, 20, 30];  
    items[index] // Panics if index out of bounds  
}
```

↳ Will it panic properly? 🤔



```
#[cfg(test)]  
mod tests {  
    use super::*;  
  
    #[test]  
    fn test_valid_index() {  
        let result = get_item(1);  
        assert_eq!(result, 20);  
    }  
  
    #[test]  
    #[should_panic]  
    fn test_invalid_index() {  
        get_item(10); // This should panic  
    }  
  
    #[test]  
    #[should_panic(expected = "index out of bounds")]  
    fn test_panic_message() {  
        get_item(100); // Panics with specific message  
    }  
}
```


Testing asynchronous

```
// Async function that can fail
async fn fetch_data(id: u32) -> Result<String, String> {
    if id == 0 {
        return Err("Invalid ID".to_string());
    }

    if id == 999 {
        return Err("Not found".to_string());
    }

    Ok(format!("Data for ID {}", id))
}
```



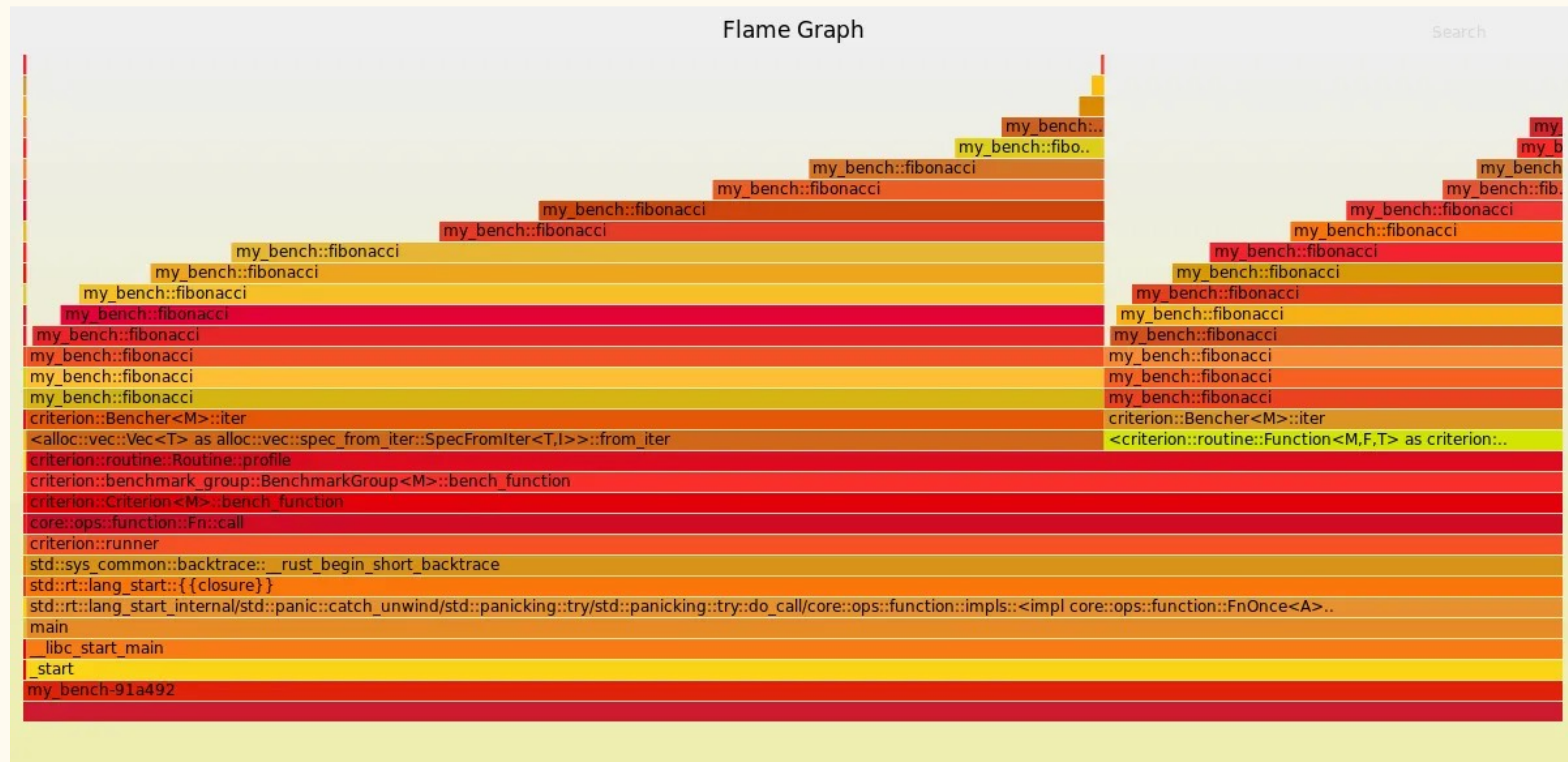
```
#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn test_fetch_success() {
        let result = fetch_data(1).await;
        assert_eq!(result, Ok("Data for ID 1".to_string()));
    }

    #[tokio::test]
    async fn test_fetch_invalid_id() {
        let result = fetch_data(0).await;
        assert!(result.is_err());
        assert_eq!(result, Err("Invalid ID".to_string()));
    }

    #[tokio::test]
    async fn test_fetch_not_found() {
        let result = fetch_data(999).await;
        assert_eq!(result, Err("Not found".to_string()));
    }
}
```

Benchmarking with Criterion



docs.rs & tests

```
/// Adds two numbers together.
///
/// # Examples
///
/// ```
/// # use D27_Testing_your_code::add;
/// let result = add(2, 3);
/// assert_eq!(result, 5);
/// ```
///
/// ```
/// # use D27_Testing_your_code::add;
/// let result = add(0, 0);
/// assert_eq!(result, 0);
/// ```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```



D27_Testing_your_code

Function `add`

Source

Search Summary

```
pub fn add(a: i32, b: i32) -> i32
```

✓ Adds two numbers together.

Examples

```
let result = add(2, 3);
assert_eq!(result, 5);
```

```
let result = add(0, 0);
assert_eq!(result, 0);
```


Let's code!

Your turn!